LEWG, SG14: P0041R0
2015-09-11
Brent Friedman
fourthgeek@gmail.com

Unstable remove algorithms

I. Summary

This proposal covers new algorithms for removing elements from a range without the stability guarantees of existing algorithms.

II. Motivation

The stability requirements of existing remove algorithms impose overhead on users, especially for types which are expensive to move. For cases where element order need not be preserved, an unstable algorithm can prove beneficial for efficiency. unstable_remove has complexity proportional to the number of elements to be removed, whereas stable removal has complexity proportional to the number of elements that need to be moved into the “holes”.

The following URL demonstrates generated assembly for implementations of similar algorithms:
https://goo.gl/xfCxzL

It is observed that unstable_remove generates less code than `remove_if` and `partition`. In particular we may note that swapping two elements, as with partition, can be much more expensive than move-assignment.

The following URL demonstrates performance tests for these same implementations:
https://github.com/WG21-SG14/SG14

It is observed that `unstable_remove_if` can outperform both `remove_if` and `partition` by a measurable degree.

These examples suggest that unstable_remove algorithms can be both smaller and faster than existing solutions.

III. Additional work

Algorithmic changes proposed in the “range” proposals should be applied to these algorithms if both are accepted.

The value of unstable_remove can be applied to containers directly, implying `unstable_erase*` algorithms or member functions. The following pseudocode signatures are informally provided here for reference and discussion but are not proposed in this paper.

```
//1.
It unstable_erase(Cont& C, It at);
//2.
It unstable_erase(Cont& C, It begin, It end);
//3.
It unstable_erase_if(Cont& C, Pred p); //unstable_remove_if + erase
//4.
It unstable_erase(Cont& C, const T& value); //unstable_remove + erase
```

IV. Discussion

Some skepticism is levied against the utility of creating unstable variants for all erase and remove features. The cost and value of each variant may be difficult to evaluate individually, which is why this proposal covers only the most fundamental functionality of `unstable_remove` and `unstable_remove_if`. This author does believe that all removal and erasure features with stability guarantees should have variants without those stability guarantees.

Some see unstable container erasure as even more important than `unstable_remove`. It is in the author’s opinion that `unstable_remove` algorithms remain independently useful in many contexts (such as fixed sized containers) and constitute more fundamental functionality than erasure.

For linked lists, the best efficiency guarantees for unstable_erase are provided by forwarding to existing, stable erase functions. It is believed that no additional wording for this case would be necessary, but some clarification may be desirable.

It is noted that for `vector<int> x`, and the following code samples,

```
x.unstable_erase( unstable_remove( x.begin(), x.end(), 0), x.end()); //A.
x.erase( unstable_remove(x.begin(), x.end(), 0), x.end()); //B.
```

A provides no efficiency benefits over B. unstable_erase provides benefits only when the range to be removed occurs within the middle of the container. This may lead to some confusion among users, though A and B would provide the same results.

V. Proposed Wording

In [alg.remove] First section:

```
template<class ForwardIterator, class T> ForwardIterator unstable_remove(ForwardIterator first, ForwardIterator last, const T& value);
template<class ForwardIterator, class Predicate> ForwardIterator unstable_remove_if(ForwardIterator first, ForwardIterator last,Predicate pred);
```

Remarks (`remove, remove_if`): Stable

Second section:

```
template<class InputIterator, class OutputIterator, class T> OutputIterator unstable_remove_copy(InputIterator first, InputIterator last, OutputIterator result, const T& value);

template<class InputIterator, class OutputIterator, class Predicate> OutputIterator unstable_remove_copy_if(InputIterator first, InputIterator last, OutputIterator result, Predicate pred);
```

Remarks (`remove_copy, remove_copy_if`): Stable